

Analiza Algoritmilor

Algorithms & Complexity Theory

Matei Popovici

Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

Many algorithms rely on recursion

Recursion requires a special treatment for determining complexity

Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

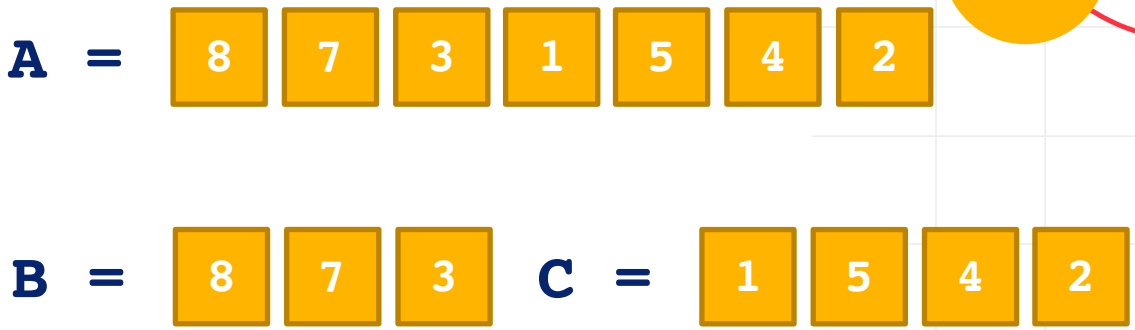


Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

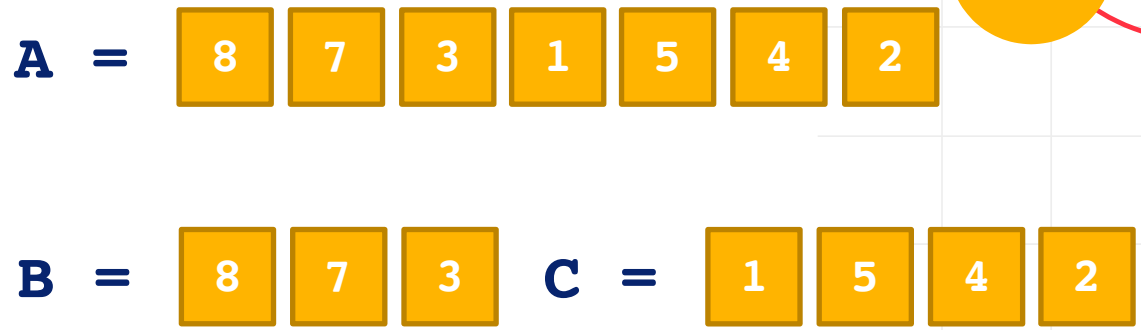


Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```



Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

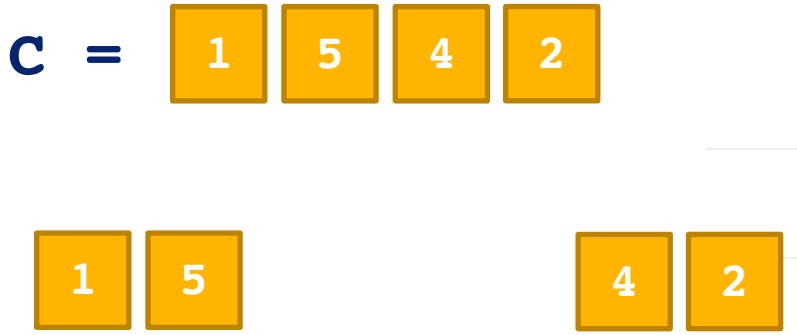
C = | 1 | 5 | 4 | 2 |

Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

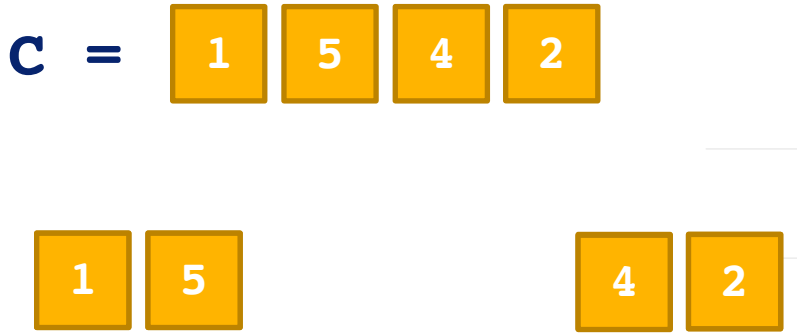


Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

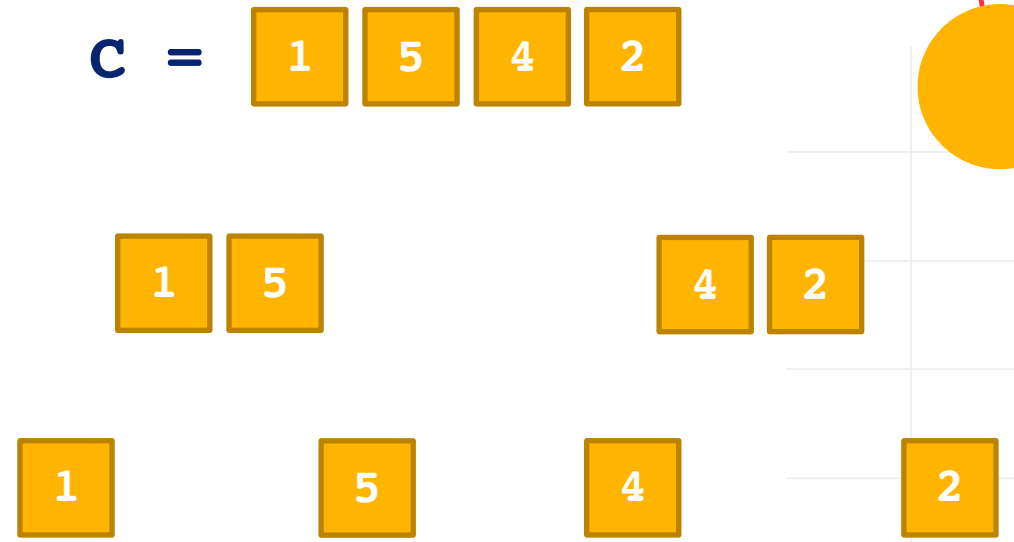


Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

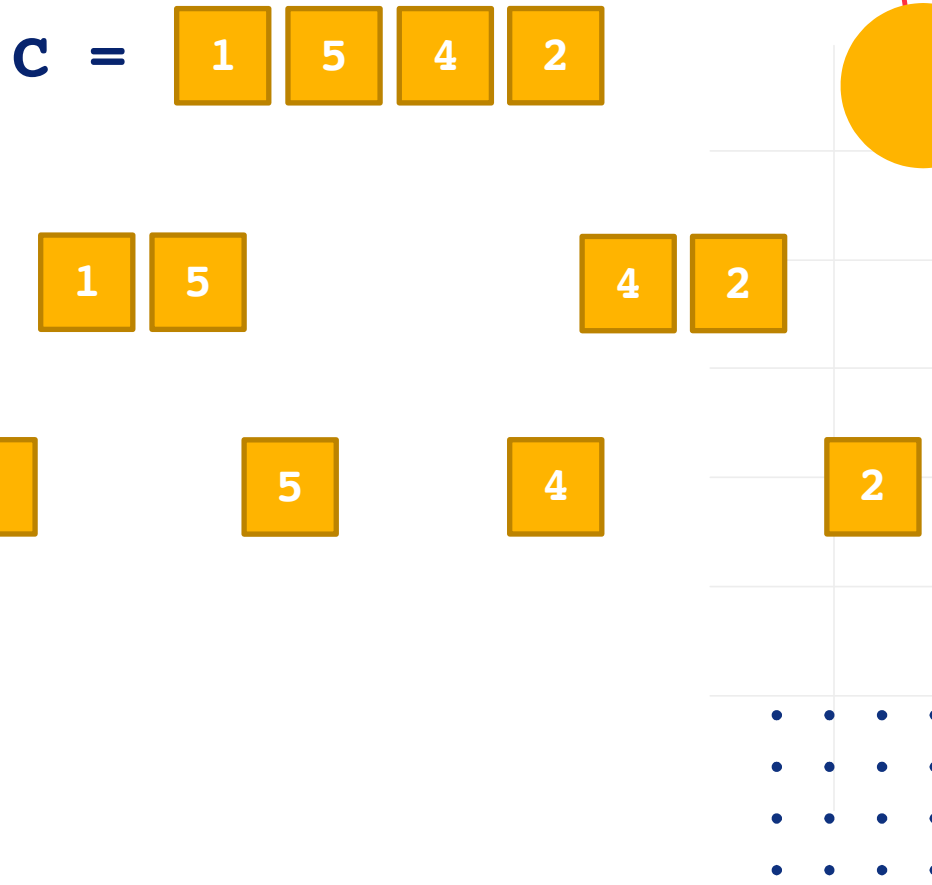


Mergesort

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

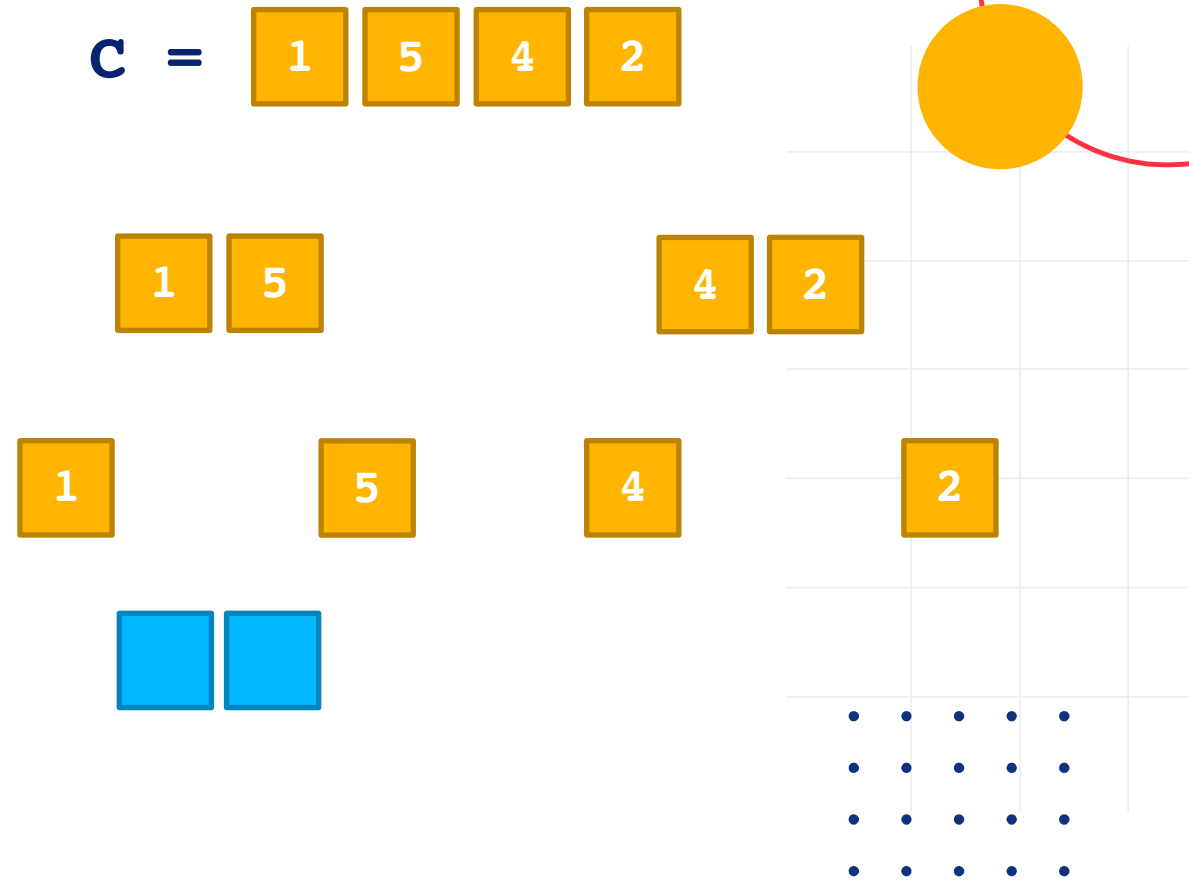
      return merge(B,C)
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

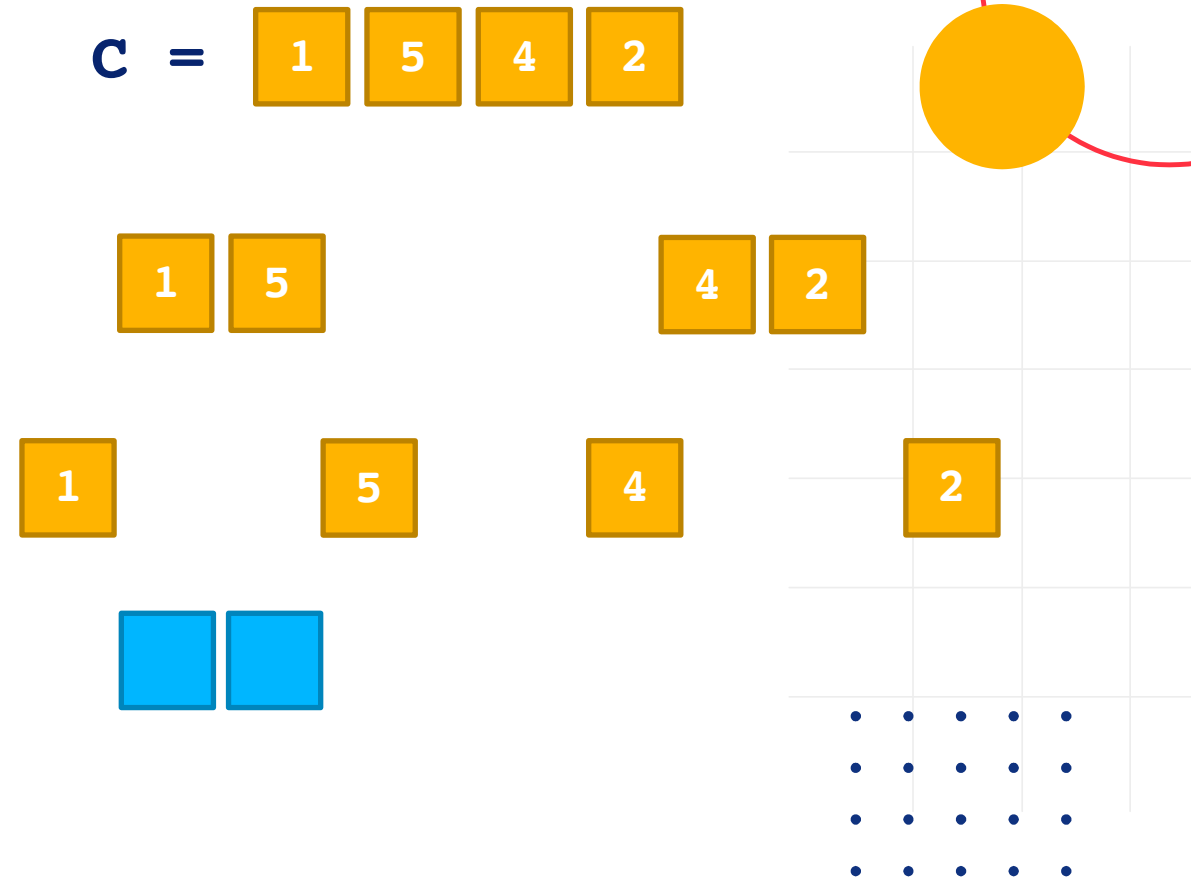
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i, j, k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

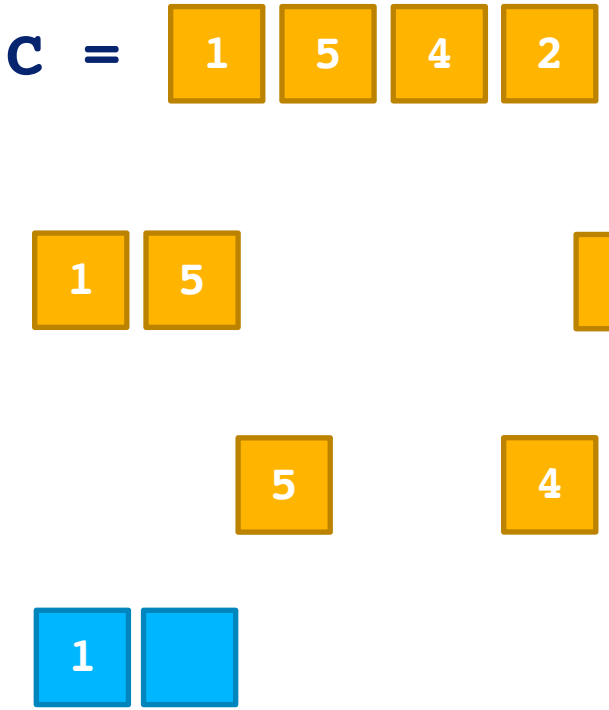
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i, j, k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```









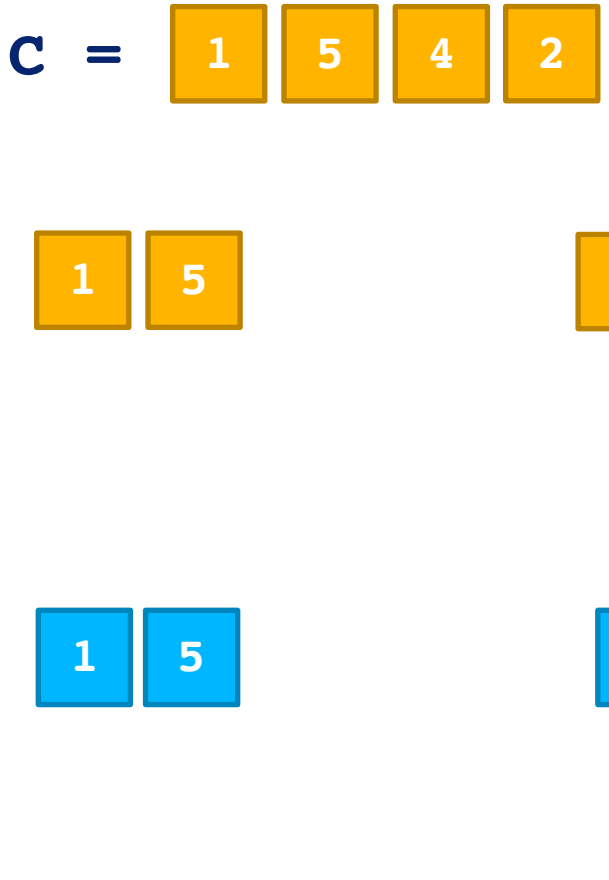




Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

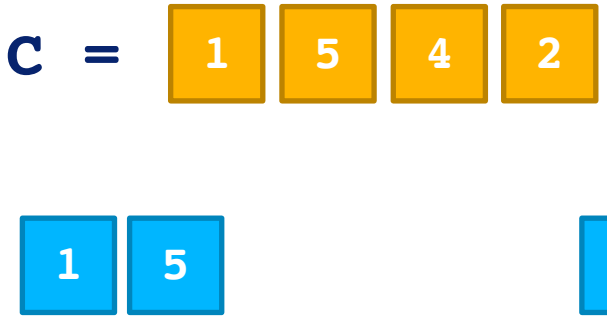
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

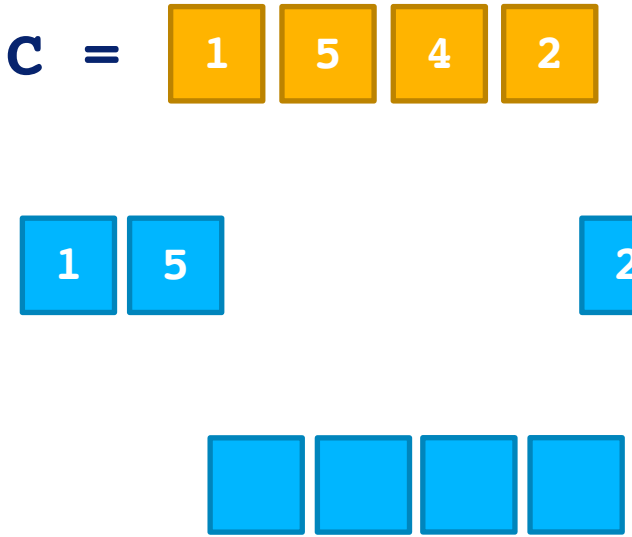
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

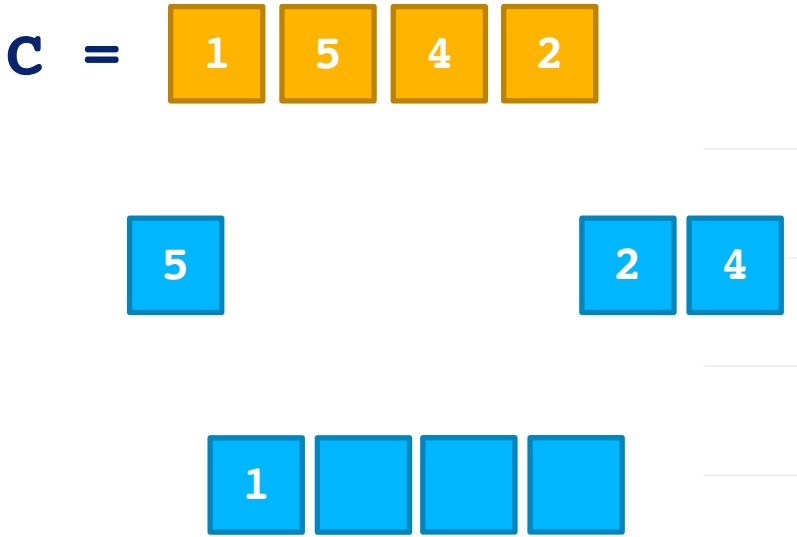
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

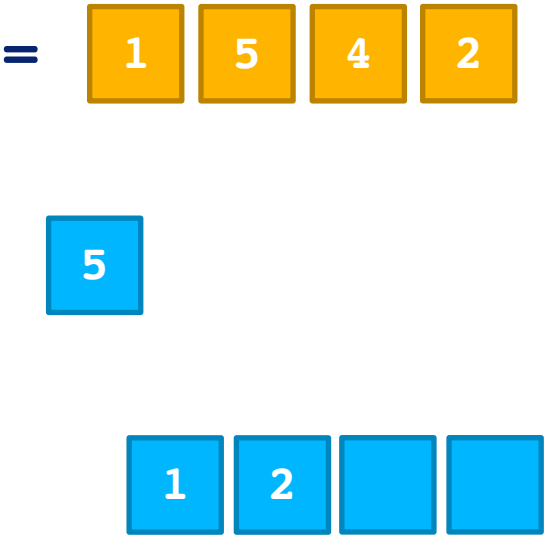
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

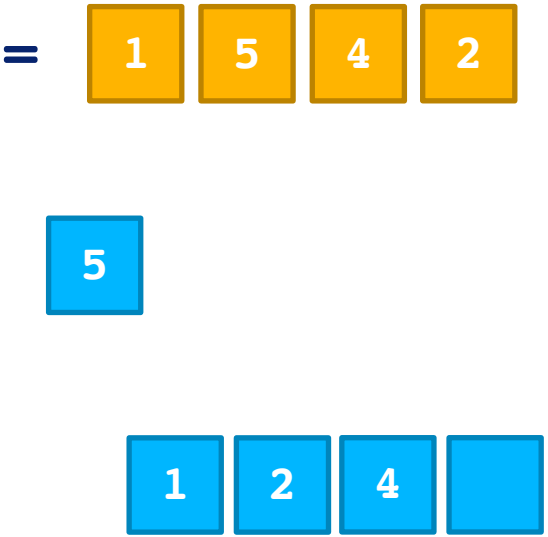
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i, j, k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```

C =

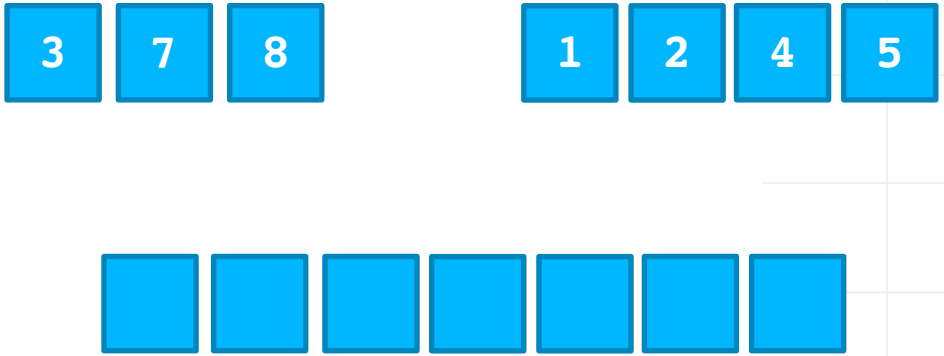




Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

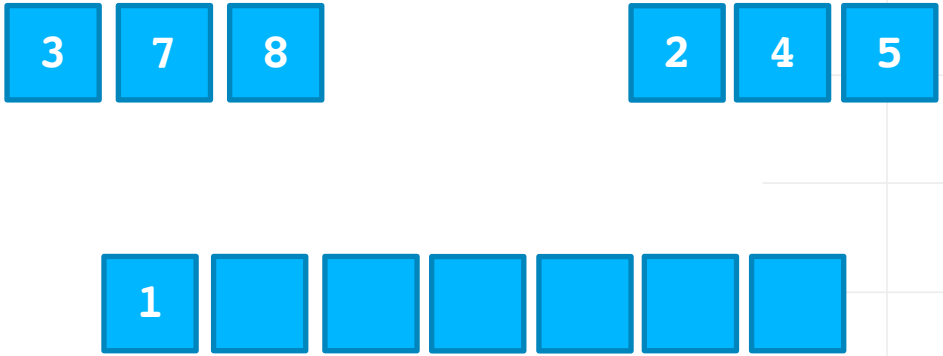
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

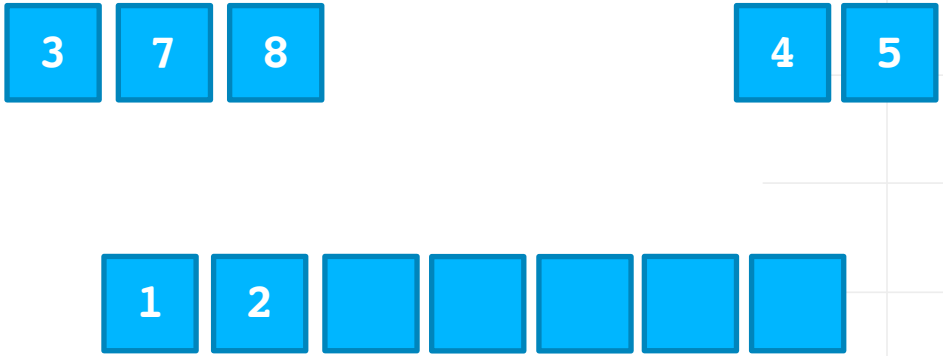
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

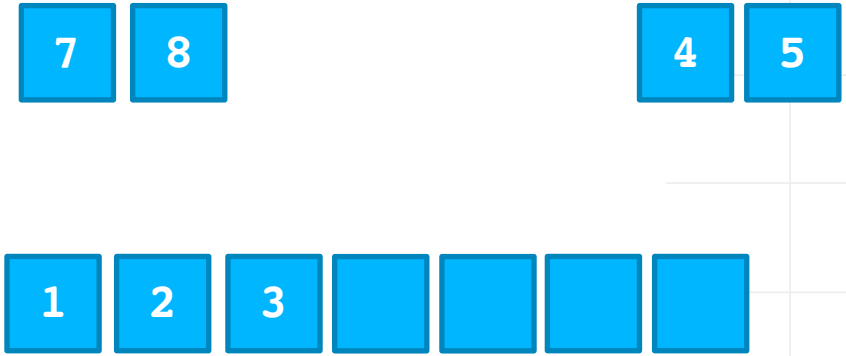
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

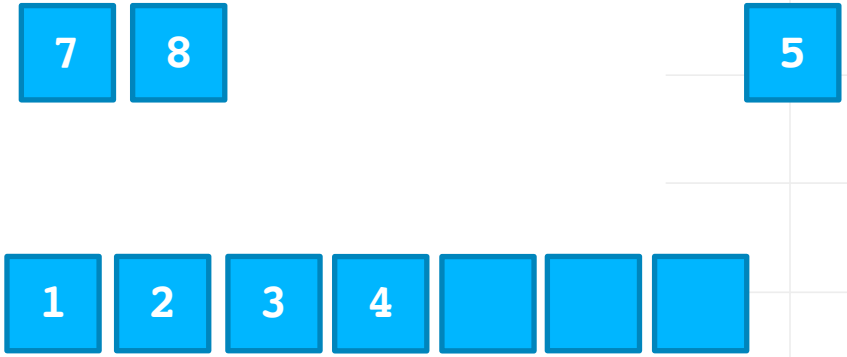
    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```

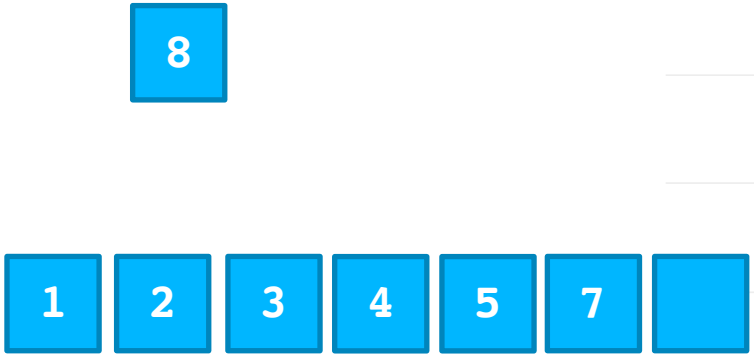




Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```



Mergesort

```
function merge (A, B)
    n := size of A
    m := size of B
    C := new array of size n + m
    i,j,k := 0
    while (j < n and k < m)
        if A[j] < B[k]
            C[i++] := A[j++]
        else
            C[i++] := B[k++]
    while (j < n)
        C[i++] := A[j++]
    while (k < m)
        C[i++] := B[k++]

    return C
```

The complexity of `merge`:

$\Theta(n+m)$

The complexity of `mergesort`:

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

$O(1)$ if n == 1

The complexity of `mergesort`:

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

$O(1)$ if n == 1

$\Theta(n)$

The complexity of `mergesort`:

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

$O(1)$ if n == 1

$\Theta(n)$

$T_{mergesort}(\lfloor n/2 \rfloor) + T_{mergesort}(\lceil n/2 \rceil)$

The complexity of `mergesort`:

```
function mergeSort(A)
    n := size of A
    if n == 1
      return n
    else
      m := n/2
      B := array of size m
      C := array of size n-m
      for i := 0,m
        B[i] := A[i]
      for j := 0,n-m
        C[j] := A[m+j]

      B := mergeSort(B)
      C := mergeSort(C)

      return merge(B,C)
```

$O(1)$ if n == 1

$\Theta(n)$

$T_{mergesort}(\lfloor n/2 \rfloor) + T_{mergesort}(\lceil n/2 \rceil)$

$\Theta(n)$

The complexity of `mergesort`:

We always assume
constant time for small n

$T_{mergesort}(n) =$

$O(1)$ if n == 1

$\Theta(n)$

$T_{mergesort}(\lfloor n/2 \rfloor) + T_{mergesort}(\lceil n/2 \rceil)$

$\Theta(n)$

The complexity of `mergesort`:

$$T_{mergesort}(n) = T_{mergesort}(\lfloor n/2 \rfloor) + T_{mergesort}(\lceil n/2 \rceil) + \Theta(n)$$



n/2 is a very good estimation for both

The complexity of `mergesort`:

$$T_{\texttt{mergesort}}(n) = 2T_{\texttt{mergesort}}(n/2) + \Theta(n)$$

The complexity of `mergesort`:

$$T_{\texttt{mergesort}}(n) = 2T_{\texttt{mergesort}}(n/2) + n$$

Let us drop the theta for a short while

Method 1 of solving the recurrences:

Recurrence trees

$$T_{mergesort}(n) = 2T_{mergesort}(n/2) + n$$

Practice:

Recurrence trees

$$T_{\text{mergesort}}(n) = 2T_{\text{mergesort}}(n/2) + n$$

Method 1 of solving the recurrences:

Substitution method

$$T_{mergesort}(n) = 2T_{mergesort}(n/2) + n$$

Method 1 of solving the recurrences:

Master method